第 3 回目　PC・マイコン・Airoco の接続とネットへのアクセス

3.1 概要

情報を統合して扱う仕組みとして，ここではNode-REDを学習する．Node-REDで，Web APIからのデータ取得，Pythonプログラムとの通信，GPIO の操作などを容易に統合することができる．ここでは，Node-RED の基本知識は既にあるものとして進める．Node-RED の知識が不足している場合は，下記リンク先などで各自で学習する．

なお，本テキストの 3.7，3.8 は発展的内容であり，時間的余裕がある者のみが実行すればよい．

参考：https://nodered.jp/docs/
https://www.youtube.com/watch?v=gEBoed0-De0

3.2 Node-RED の Jetson Nano へのインストール

Jetpack の「ソフトウェア」アイコンからも Node-RED をインストールできるが，バージョンが合わなくて Node-RED から GPIO 操作ができなかった（2025.4.2 時点）．そこで，動作確認ができている Node-RED と，それを動かすための Node.js のバージョンを以下のコマンドでインストールする．（PDF ファイルから Copy&Paste するときには文字「-」の文字化けに注意する．）

Node.js のインストール：

```
$ sudo apt install -y curl
$ mkdir ~/work
$ export XDG_CONFIG_HOME=~/work
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
$ export NVM_DIR="$XDG_CONFIG_HOME/nvm"
$ source "$NVM_DIR/nvm.sh"
$ nvm install 17.9.1
```

Node-RED のインストール：

```
$ npm install -g --unsafe-perm node-red@3.0.2 node-red-admin@3.0.0
```

ウォーニングメッセージが出るが一応インストールできる．2025 年 4 月 17 日時点のインストールでは，Fig. 1 の表示が出てインストールが終了した．

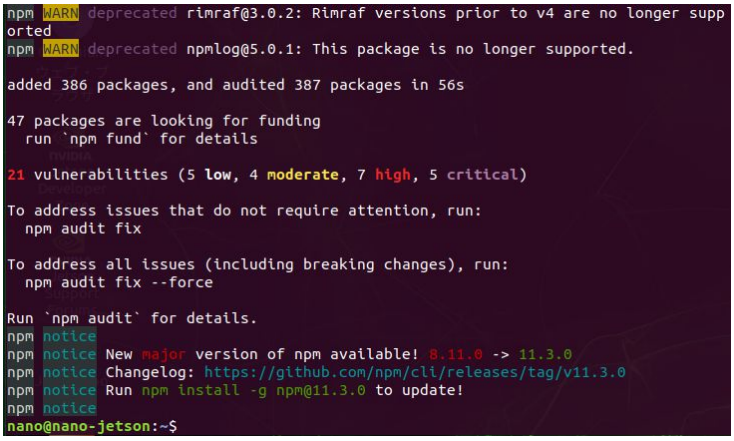

Fig. 1　Node-RED インストール時の画面表示

ターミナルで

```
$ node-red
```

とすることで，Node-RED を実行する．同一 Jetson 内の Web ブラウザで 127.0.0.1:1880 を開いて Node-RED に接続すると GUI が表示される．LAN で繋がっている別の PC から，Jetson の IP アドレスのポート 1880 を開いても Jetson の Node-RED の操作が可能である．

3.3 物体検出アプリの Node-RED への接続

カメラからのリアルタイム物体検出アプリ「Tiny YOLO v2」の検出結果を Node-RED に伝える．

Node-RED のプログラム（フロー）を作成する．Node-RED では，フローを作るノードが左カラムに表示される．「共通」欄の「inject」「debug」と「ネットワーク」欄の「mqtt in」ノードを右側のスペースにドラッグ＆ドロップし，Fig. 2 のように繋げる．

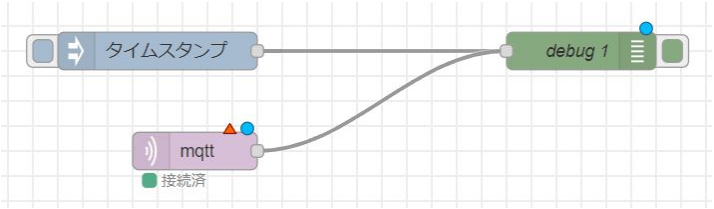

Fig. 2 「mqtt in」ノードの接続例

ノードについている赤い三角マークは，設定が未完了であることを表している．また，右側サイドバーの「i」アイコンを押すことで，ヘルプ情報が出る．「mqtt in」ノードをダブルクリックして編集画面を開き，サーバの鉛筆アイコンをクリックすると「プロパティ」が開くので，「接続」タブのサーバに Jetson の IP アドレス（127.0.0.1 で良い）を入力する（Fig . 3）．「追加」ボタンを押し，「トピック」を記入し「完了」ボタンを押すことで設定が完了する．ここで決めたトピックは，後の接続で必要になる．

Fig. 3 「mqtt in」ノードのプロパティ画面

「デプロイ」して，デバッグタブを表示（画面右の虫アイコンをクリック）し，タイムスタンプ横のボタンをクリックすることで，

デバッグタブにタイムスタンプが表示されることを確認する．これで Node-RED のフローは完成した．なお，デプロイ後に停止するには，右上ハンバーガメニュー（横三本線のアイコン）から「フロー｜フロー名を変更」を選び，立ち上がったプロパティタブの「有効」ボタン（最下部にある）をクリックして「無効」にしてから再度デプロイすれば良い（Fig. 4）．

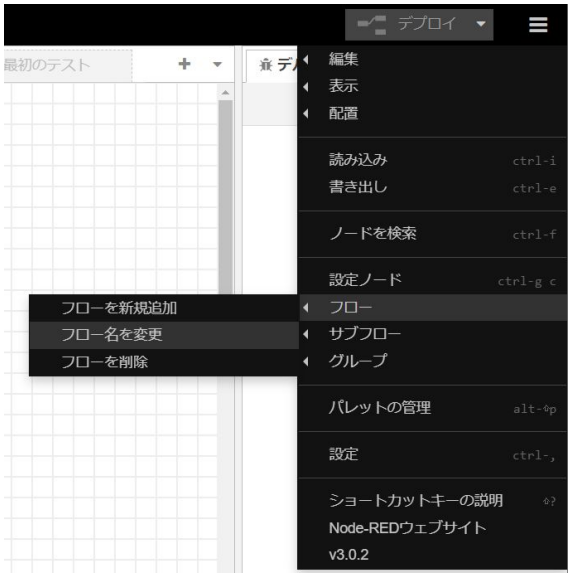

Fig. 4 フローの停止方法

「Tiny YOLO v2」で検出した情報を MQTT という通信方式で MQTT ブローカーに送り，MQTT ブローカーはさらに，MQTT で Node-RED に情報を送る．そのために，MQTT ブローカーである「mosquitto」をインストールする．

```
$ sudo apt install mosquitto
```

次に，Jetson のパワーモードを最大クロックにしてから，以下を実行する．[トピック名]は，さきほど決めたもので置き換える．

```
$ cd ~/tiny_yolov2_onnx_cam
$ python3 tiny_yolov2_onnx_cam_mqtt.py --host 127.0.0.1 --topic [トピック名] --port 1883
```

「Tiny YOLO v2」の検出結果が，Node-RED のデバッグタブに表示されることを確認する（Fig. 5）．

Fig. 5 「Tiny YOLO v2」による検出結果表示例

3.4 WebAPI への接続（Airoco への接続）

Airoco へのアクセスを例に，一般の WebAPI 利用方法を説明する．Fig. 6 のようなフローを組む（フローを追加するには，右上の「+」ボタンを押せば良い）．「http request」ノードをダブルクリックして現れる「プロパティ」の「URL」に，Airoco

の全センサデータを JSON で取得するための URL を設定し，さらに「出力形式」として「JSON オブジェクト」を選択すると，出力として JSON が返ってくる．例えばセンサ 3 の CO2 値を取り出すために，「function」ノードに以下のようなプログラムを書き込む．タイムスタンプ横のボタンを押して，Debug ウインドウに CO2 値が表示されるのを確認する．

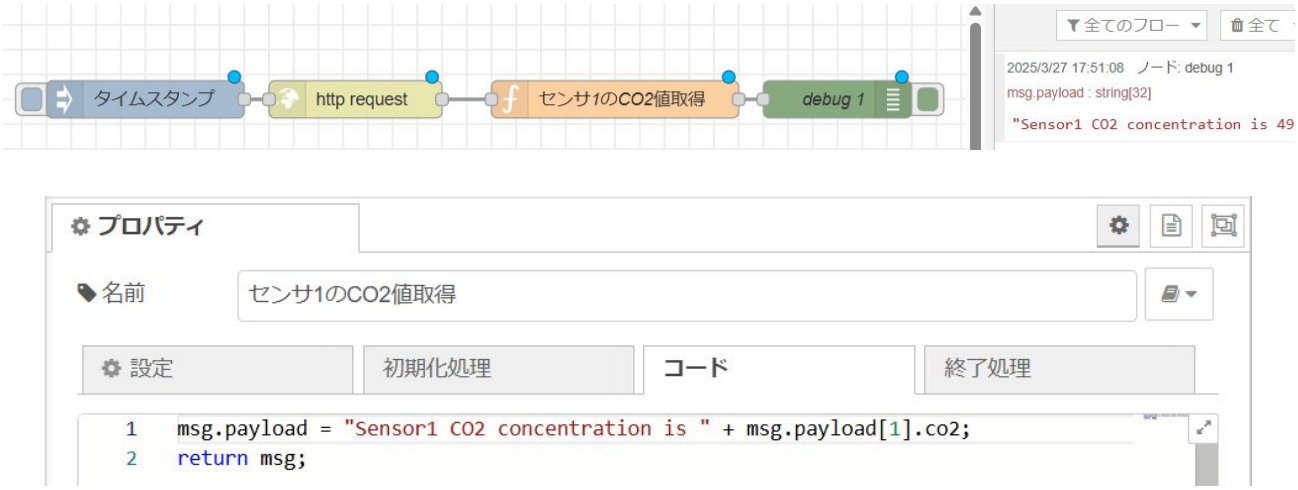

Fig. 6　Airoco から出力を得るフロー

3.5 天気情報の取得

Node-RED には，特定の WebAPI 用の専用モジュールが用意されている場合がある．ここでは，OpenWeatherMap から天気情報を取得する例を見る．Node-RED 右上のハンバーガメニューから「パレットの管理」を選択後，「ノードを追加」タブを選択し，検索欄を使って「node-red-node-openweathermap」を探して「ノードを追加」ボタンを押す．インストール後に，パレットに「weather」というグループが現れるのを確認する．

新規フローを用意し，パレットの「weather」グループから「openweathermap」ノードをワークスペースにドラッグ＆ドロップする．「inject」と「debug」ノードもドラッグ＆ドロップし，Fig. 7 のようなフロー組む．

Fig. 7　OpenWeatherMap に接続するフロー

「openweathermap」を使用するためには「API Key」を取得する必要がある．

https://openweathermap.org/

にアクセスし，アカウントを取得する．ログイン後，右上の自分の名前横の下三角形をクリックして「My API keys」を選択すると API キー作成用画面が現れる．デフォルトキーか新たに作成したキーを用いる（作成した API Key が有効になるまで多少時間がかかることがある）．これを Node-RED の「openweathermap」ノードのプロパティの「API Key」にコピー&ペーストする．「openweathermap」ノードの「Language」や「Location」も適切に設定する．タイムスタンプ横のボタンを押すことで，サイドバーのデバッグタブに天気の情報が表示されるか確認する．Fig. 8 に表示例を示す．

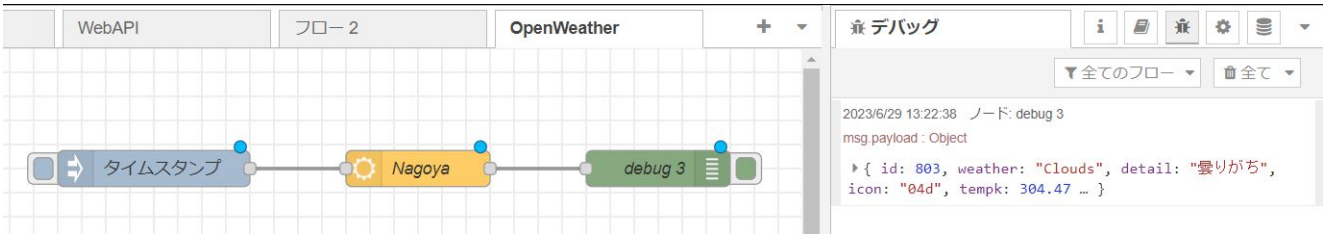

Fig. 8 OpenWeatherMap の出力例

参考: https://tool-lab.com/raspberrypi-node-red-34/

3.6 Node-RED からの GPIO 操作

まず, Node-RED から GPIO を操作するためのモジュールをインストールする. Node-RED 右上のハンバーガメニューから「パレットの管理」を選択後, 「ノードを追加」タブを選択し, 検索欄を使って「node-red-node-pi-gpio」を探して「ノードを追加」ボタンを押す. インストール後に, パレットに「Raspberry Pi」というグループが現れるのを確認する.

Raspberry Pi から Jetson 対応に変更するため, ~/.node-red/node_modules/node-red-node-pi-gpio ディレクトリにあるファイル nrgpio.py と testgpio.py の最初の

```
import RPi.GPIO as GPIO
```

を

```
import Jetson.GPIO as GPIO
```

に変更する.

さらに, GUI で入出力を変更できるように, Dashboard をインストールする. 「パレット管理」の「ノードを追加」タブを選択後, 検索欄で「node-red-dashboard」を探して「ノードを追加」ボタンを押す. インストール後に, パレットに「dashboard」というグループが現れるのを確認する.

第 2 回の LED 点滅の回路を操作するフローを組む. 「+」ボタンを押してフローを追加する. パレットの「dashboard」グループから, 「switch」を選択し, ワークスペースにドラッグ&ドロップする. このノードをダブルクリックして「プロパティ」タブの「Group」の鉛筆アイコンをクリックして, さらに現れた「タブ」の鉛筆アイコンをクリック後, 「追加」ボタンを 2 回押して, 最後に「完了」ボタンを押す. 「Raspberry Pi」グループから, 「rpi-gpio out」を選択し, ワークスペースにドラッグ&ドロップする. このノードをダブルクリックして, 端子 12 を選択(Fig. 9)し, さらに「端子の状態を初期化」にチェックを入れて「端子の初期レベル-Low(0)」が選択されているのを確認して「完了」を押す. 「switch」ノードの出力と「rpi-gpio out」ノードを接続する. 前回と同様に GPIO 端子 12 で LED を操作できるように回路を作成し, このフローを「デプロイ」する. Node-RED のサーバの URL の後ろに「ui」をつける(例: 127.0.0.1:1880/ui)と, マウスで操作できるスイッチが現れる(Fig. 10)ので, LED がオン/オフできることを確認する(UI は, Fig. 11 に示すようなサイドバーの Dashboard タブの「四角に右上矢印」アイコンでも出すことができる.). 「dashboard」を使うと, グラフ表示なども可能である(Fig. 12). 見栄えのする表示画面も作れるので, 興味のある人は調べてみよう.

Fig. 9 「rpi-gpio out」ノードで端子 12 を選択

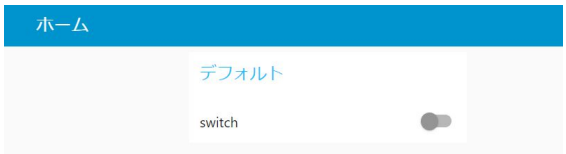

Fig. 10 「switch」ノードで作成されたスイッチ

Fig. 11 サイドバーダッシュボードタブの「四角に右上矢印」

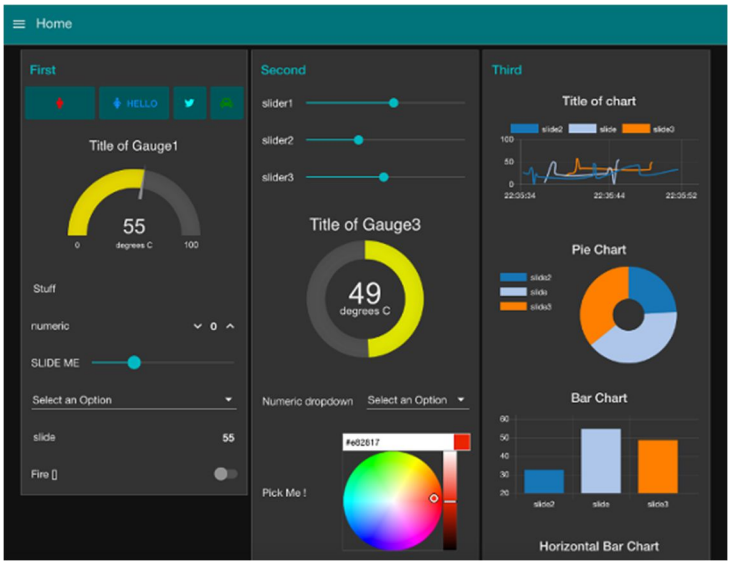

Fig. 12 **dashboard による GUI 作成例**（出典：https://flows.nodered.org/node/node-red-dashboard）

参考： https://tool-lab.com/raspberrypi-node-red-23/
https://node-red.ia-cloud.com/dashboard-basic/about-dashboard
https://qiita.com/takeyan/items/fff5e8d64da25220a0c0

3.7 Python プログラムとの通信【発展的内容】

Python プログラムと Node-RED の間のデータのやり取りとして，ここでは HTTP リクエストの例を見る．「http in」「change」「http response」「debug」ノードで，Fig. 13 のフローを組む．

Python で JSON データを作成して POST，Node-RED 側で POST を受信した結果を Debug ウィンドウに表示し，”Ok”というレスポンスを返している．フローを「デプロイ」後，ターミナルで以下のコマンドで sample3-1.py（Fig. 14）を実行することで通信ができていることを確認する．

```
$ python3 sample3-1.py
```

実行後の出力を Fig, 15 に示す。

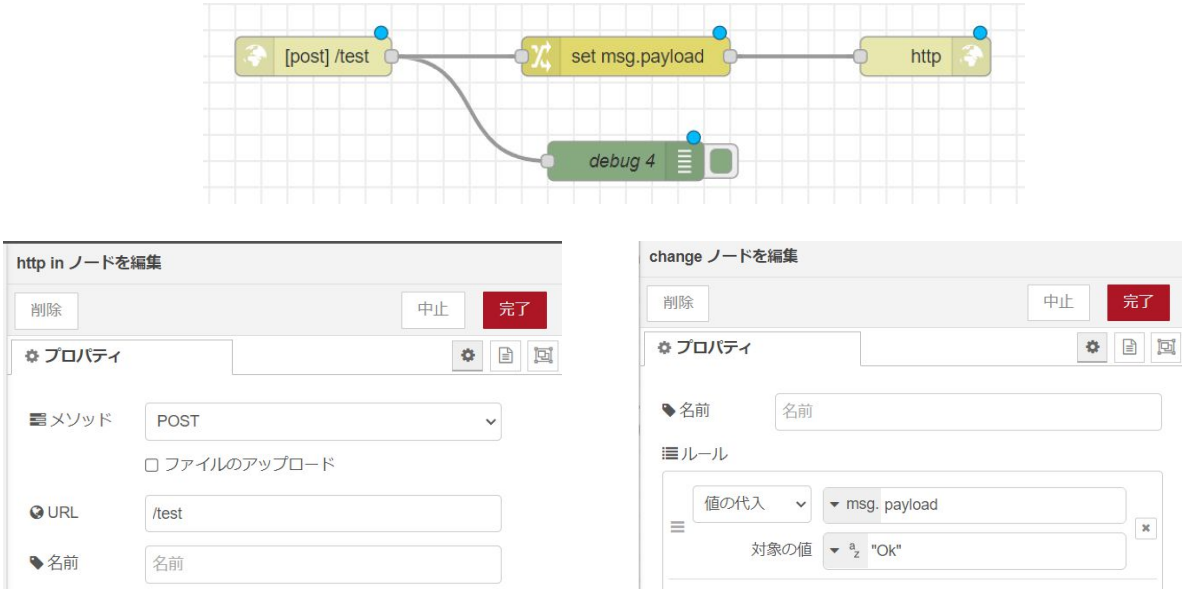

Fig. 13 Python プログラムと通信するフロー

```python
import urllib.request, json

if __name__ == '__main__':
    url = "http://localhost:1880/test"
    method = "POST"
    headers = {"Content-Type" : "application/json"}

    # PythonオブジェクトをJSONに変換する
    obj = {"Hello" : "world", 123 : 456}
    json_data = json.dumps(obj).encode("utf-8")

    # httpリクエストを準備してPOST
    request = urllib.request.Request(url, data=json_data, method=method, headers=headers)
    with urllib.request.urlopen(request) as response:
        response_body = response.read().decode("utf-8")
        print(response_body)
```

Fig. 14 sample3-1.py

Node-RED のデバッグ出力

```
nano@nano-jetson:~/Public$ python3 sample3-1.py
"Ok"
```

Python 側の実行結果

Fig. 15　Python と Node-RED の通信の結果

3.8 課題

【基本課題 3-1】 Node-RED を使って，1 秒ごとに LED をオン/オフするフローを作成し実行せよ．（例：inject ノードで周期信号作成，delay ノードで一定時間遅れた信号を作成，change ノードで遅延した信号から値に変換する）